

Linear Regression

Dept. of CSE, University of Asia
Pacific

Linear

Regression

Linear Regression is a supervised learning algorithm used to **predict continuous numerical values** by learning the relationship between one or more independent variables and a dependent variable. It assumes that this relationship can be approximated using a straight line.

It models the relationship between:

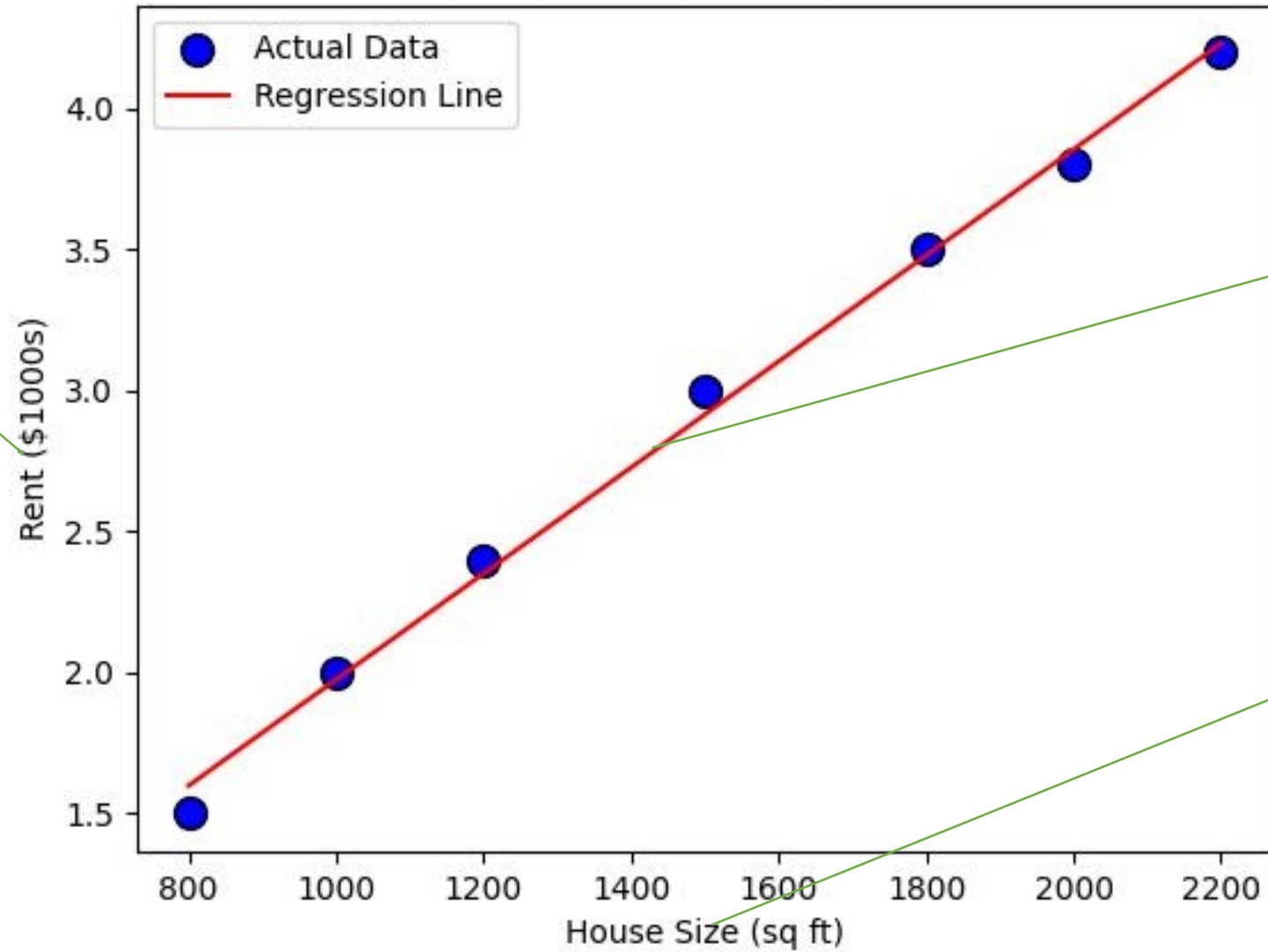
- Independent variable(s) → input/features
- Dependent variable → target/output

using a linear function.

Examples

- House size → Rent
- Study hours → Exam marks
- Advertising expenditure → Sales
- Engine size → Fuel consumption
- Temperature → Energy consumption

House Size vs Rent Prediction



Dependent Variable

Regression Line

Independent Variable

Pros and Cons

Pros

- Simple model
- Computationally efficient
- Interpretability of the Output

Cons

- Overly-Simplistic
- Linearity Assumption
- Severely affected by Outliers

Basic Linear Regression

Idea

The primary objective of Linear Regression is to determine the **best-fitting straight line** that minimizes the difference between the actual target values and the predicted target values.

- Instead of connecting all data points, the algorithm searches for a line that produces the **smallest overall p**

Mathematical Model

The hypothesis function is

$$\hat{y} = wx + b \quad \text{Or} \quad \hat{y} = b_0 + b_1x$$

$w = b_1$
 $b = b_0$

where

Symbol

Meaning

x

Independent variable (feature)

$\hat{y}$

Predicted output

w

Weight (slope of the line)

b

Bias (intercept)

Basic Linear Regression Idea

Example

$$Rent = b_0 + b_1(Size)$$

If

$$Rent = 0.5 + 0.002(Size)$$

then for a 1,500 sq. ft. house:

$$\begin{aligned} Rent &= 0.5 + 0.002(1500) \\ &= 3.5 \end{aligned}$$

Therefore, predicted rent = **\$3,500** if rent is measured in \$1,000s.

How Does Linear Regression Find the Best Line?

Suppose we have several observations:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

Many possible lines can be drawn.

The goal is to find the line that produces the **smallest prediction errors**.

For each observation:

$$e_i = y_i - \hat{y}_i$$

where:

- y_i = actual value
- $\hat{y}_i$ = predicted value
- e_i = residual/error

Mean Squared Error

The Mean Squared Error is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Example

Suppose residuals are:

$$2, -3, 1, -2$$

Squared errors:

$$4, 9, 1, 4$$

Therefore:

$$\begin{aligned} MSE &= \frac{4 + 9 + 1 + 4}{4} \\ &= \frac{18}{4} \\ &= 4.5 \end{aligned}$$

Goal of Linear Regression

Minimize MSE

Best-Fit Line

Linear regression finds:

$$\hat{y} = b_0 + b_1x$$

such that the **sum of squared errors** is minimized.

This is called the:

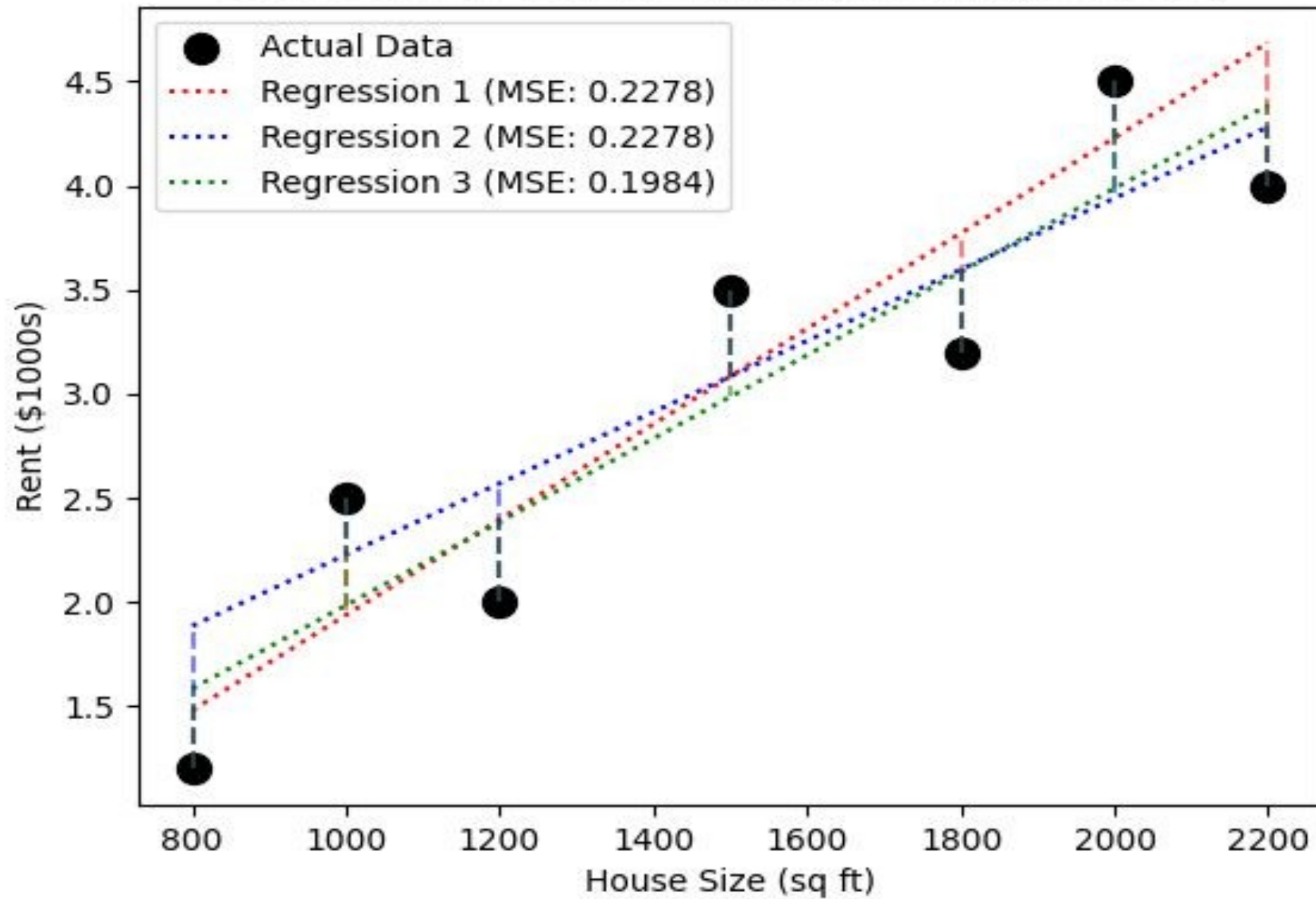
Ordinary Least Squares (OLS)

Objective:

$$\min_{b_0, b_1} \sum_{i=1}^n (y_i - b_0 - b_1x_i)^2$$

The resulting line is called the **best-fit regression line**.

House Size vs Rent Prediction (Best: Regression 3)



Calculating the Slope Manually

For simple linear regression:

$$b_1 = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2}$$

Then:

$$b_0 = \bar{y} - b_1 \bar{x}$$

where:

$$\bar{x} = \frac{\sum x}{n}$$

and

$$\bar{y} = \frac{\sum y}{n}$$

Using the Model for Prediction

$$\hat{y} = b_0 + b_1x$$

If $b_0 = 1.8$, $b_1 = 0.8$

Suppose a student studies:

$$x = 6 \text{ hours}$$

Then:

$$\hat{y} = 1.8 + 0.8(6)$$

$$= 1.8 + 4.8$$

$$\boxed{\hat{y} = 6.6}$$

The model predicts **6.6 marks/units**.

Gradient Descent

Gradient Descent is an optimization algorithm used to find the values of w and b that minimize the cost function.

From Prediction to Optimization

We have the linear regression model:

$$\hat{y} = wx + b$$

For each training example:

$$\text{Error} = y - \hat{y}$$

The model should choose w and b so that the prediction errors are as small as possible.

Objective

We measure the overall prediction error using Mean Squared Error (MSE):

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Since

$$\hat{y}_i = wx_i + b$$

we can write:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2$$

Gradient Descent: Complete Training Process

Algorithm

1. Initialize parameters

$$w, b \leftarrow \text{initial values}$$

2. Choose learning rate

$$\alpha > 0$$

3. Predict

$$\hat{y}_i = wx_i + b$$

4. Calculate cost

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2$$

5. Calculate gradients

$$\frac{\partial J}{\partial w}, \quad \frac{\partial J}{\partial b}$$

6. Update parameters

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}$$

$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

7. Repeat

Continue until:

- MSE becomes sufficiently small, or
- parameter changes become very small, or
- maximum number of iterations is reached.

Example

Suppose we have one training point:

$$x = 2, \quad y = 5$$

Initialize:

$$w = 1, \quad b = 0$$

Learning rate:

The **learning rate** α controls how large each step is.

$$\alpha = 0.1$$

Step 1 — Prediction

$$\hat{y} = wx + b$$

$$\hat{y} = (1)(2) + 0 = 2$$

The model moved from predicting **2** $\rightarrow$ **5**, reducing the error from **3** $\rightarrow$ **0** for this simple example.

In a real dataset, the gradients are calculated using **all training examples**, and the process is repeated.

Step 2 — Calculate Error

$$e = y - \hat{y}$$

$$e = 5 - 2 = 3$$

Step 3 — Calculate Gradients

For one observation:

$$\frac{\partial J}{\partial w} = -2xe$$

$$= -2(2)(3) = -12$$

and

$$\frac{\partial J}{\partial b} = -2e = -6$$

Step 4 — Update Parameters

$$w_{\text{new}} = 1 - (0.1)(-12) = \boxed{2.2}$$

$$b_{\text{new}} = 0 - (0.1)(-6) = \boxed{0.6}$$

New Model

$$\hat{y} = 2.2x + 0.6$$

For $x = 2$:

$$\hat{y} = 2.2(2) + 0.6 = 5.0$$



GRADIENT DESCENT

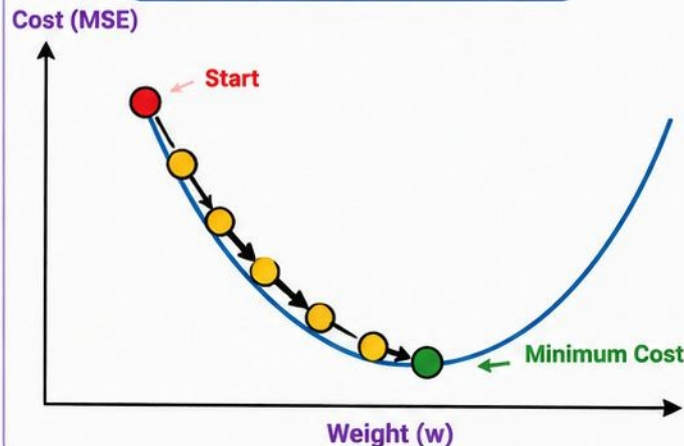


Gradient Descent is an optimization algorithm used to minimize the cost function by iteratively updating the parameters in the **opposite direction** of the gradient.

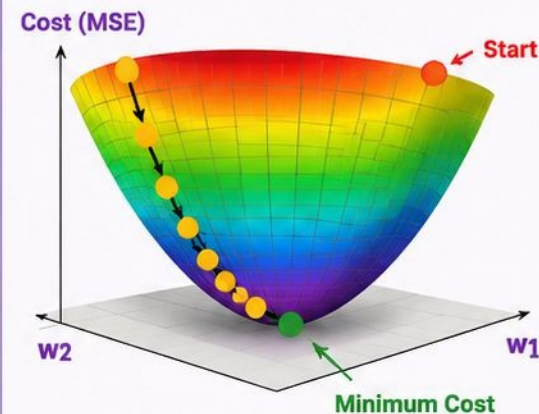
THE IDEA

- Start with an initial value of the parameter (w).
- Compute the slope (gradient) of the cost function.
- Update w in the **opposite** direction of the slope.
- Repeat until we reach the **minimum** cost.

COST (MSE) vs WEIGHT (w)



3D VIEW (Cost Surface)



HOW IT WORKS (STEP BY STEP)

1 Choose an initial weight (w).

2 Calculate the gradient
 $\left(\frac{dJ}{dw}\right)$

3 Update the weight:
$$w_{new} = w_{old} - \alpha \frac{dJ}{dw}$$

where α is the learning rate.

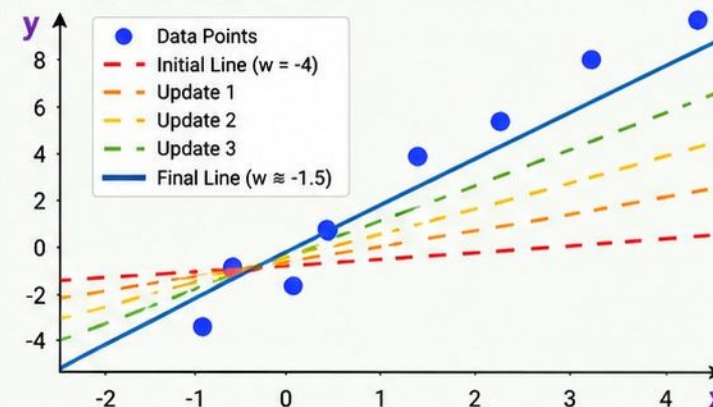
4 Repeat until convergence.

Iteration	w	Cost (MSE)	Gradient $\left(\frac{dJ}{dw}\right)$	Update (w_{new})
0	-4.00	25.000	-10.000	-3.00
1	-3.00	9.000	-6.000	-2.40
2	-2.40	3.240	-3.600	-2.04
3	-2.04	1.166	-2.160	-1.82
4	-1.82	0.420	-1.296	-1.69
5	-1.69	0.151	-0.778	-1.61
...	...	...	...	...
Converges to	-1.50	≈ 0 (Minimum)	≈ 0	-1.50

Learning Rate (α) = 0.1 | Optimal Weight (w^*) $\approx$ -1.50

EXAMPLE: LINEAR REGRESSION

Goal: Find the best line ($y = wx + b$) that minimizes the **MSE**.



Gradient Descent adjusts w (and b) step by step to find the line with the **minimum cost (MSE)**.

Multiple Linear Regression

With multiple predictors:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \cdots + b_px_p$$

Example:

$$Rent = b_0 + b_1(Size) + b_2(Bedrooms) + b_3(Location)$$

Multiple vs. Multivariate Regression

Simple Linear Regression

1 predictor $\rightarrow$ 1 target

Multiple Linear Regression

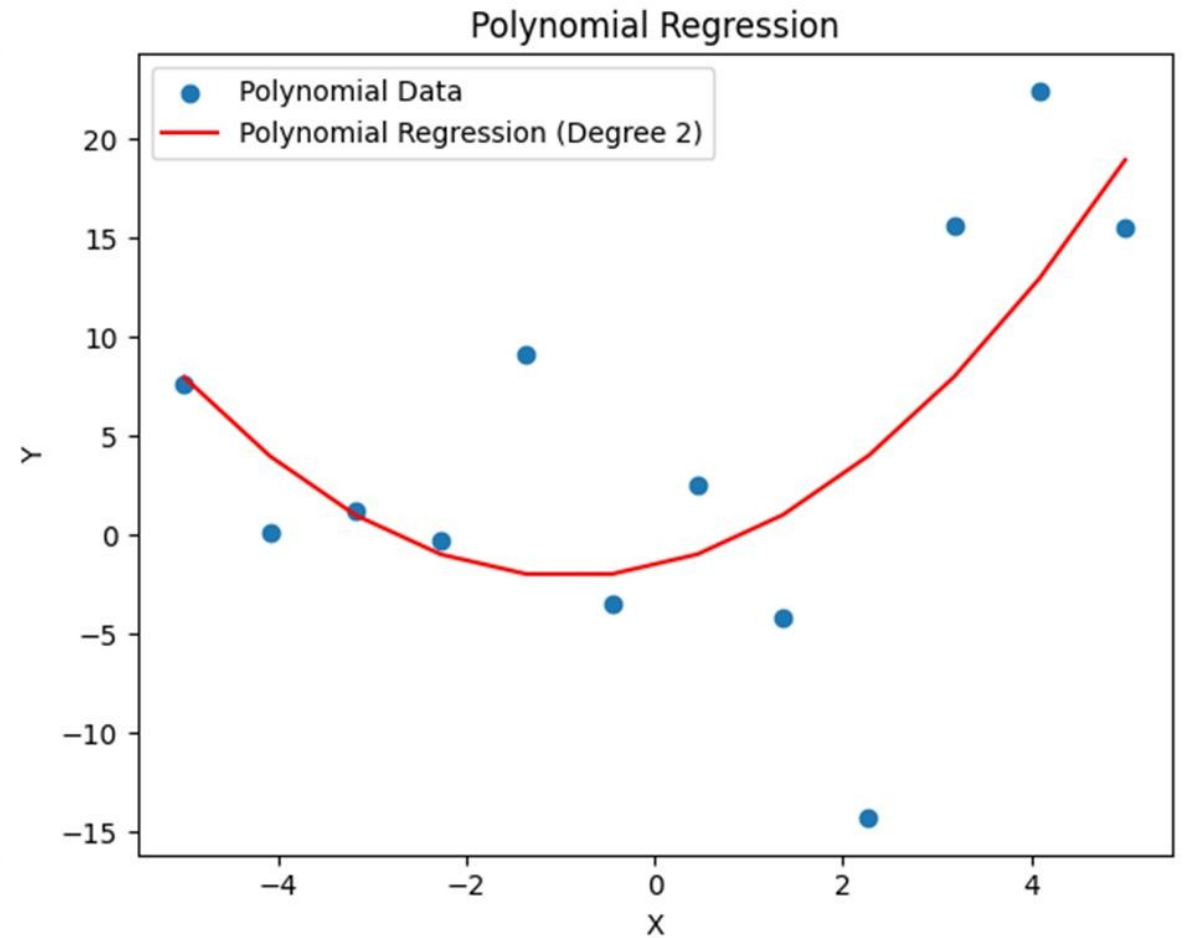
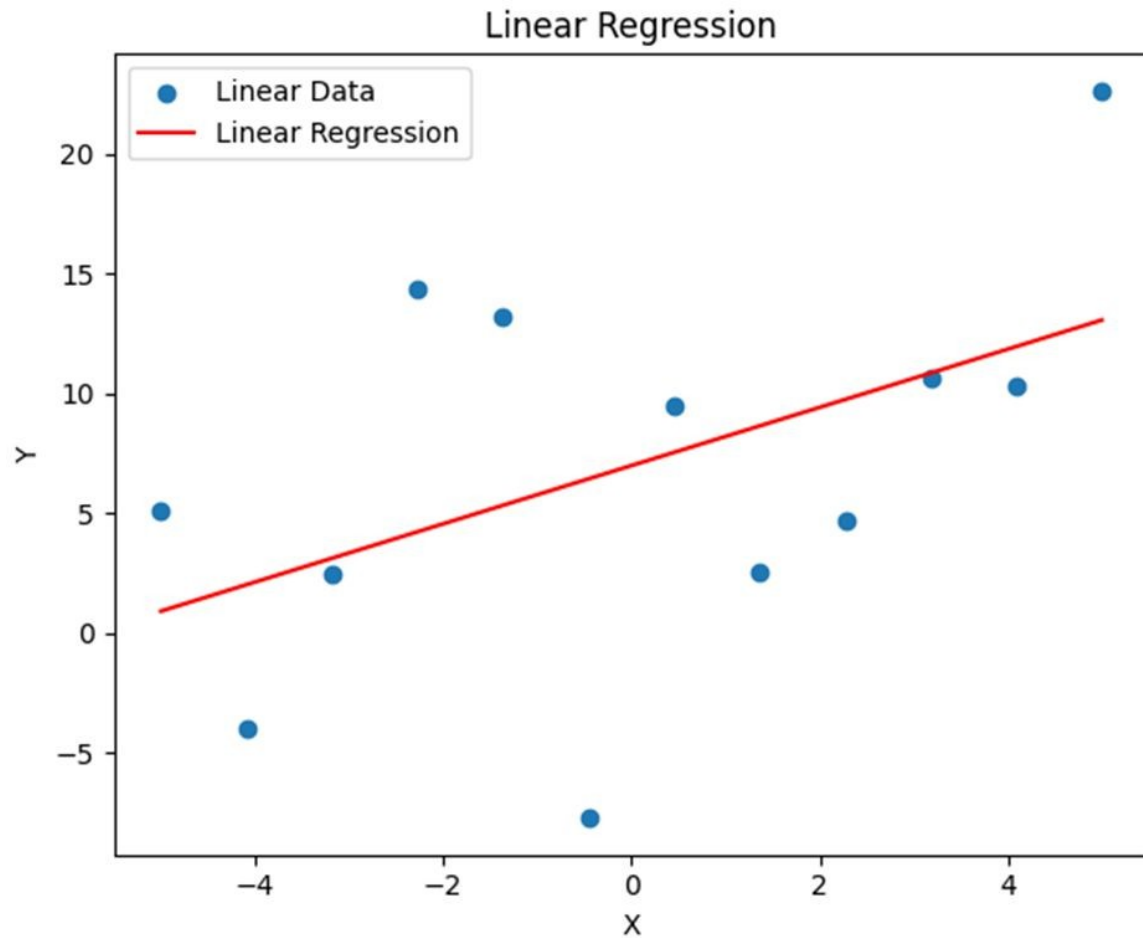
2 + predictors $\rightarrow$ 1 target

Multivariate Regression

1 + predictors $\rightarrow$ 2 + targets

Multivariate regression is a statistical technique used to predict **more than one dependent (output) variable** at the same time using one or more independent (input) features

Polynomial Regression



Pros and Cons

Pros

- The best approximation of the connection between the dependent and independent variables
- Can accommodate a wide range of functions

Cons

- One or two outliers in the data might have a significant impact
- Fewer model validation methods for detecting outliers in nonlinear regression